

NagarMitra - System Design Document

1. Data Model (Database Schema)

The database is built on a relational model using SQLite and managed via Prisma ORM. The core entities are linked via foreign keys to maintain strict data integrity.

Key Entities:

- **User:** The central identity model.
 - *Fields:* id, name, email, password_hash, role, department_id.
 - *Relationships:* Can have multiple filed complaints (if Citizen), assigned complaints (if Field Staff), and audit logs.
- **Department & Category:** Defines the organizational structure.
 - *Department Fields:* id, name, head_id.
 - *Category Fields:* id, name, department_id, sla_hours. (Categories dictate the expected resolution timeframe).
- **Complaint:** The core transactional record.
 - *Fields:* id, description, location_text, priority, status, deadline.
 - *Staff Action*
Fields: staff_visit_date, staff_contact, staff_message, dept_warning.
 - *Relationships:* Belongs to one Citizen, one Category, and optionally one Assignee (Field Staff). Contains multiple Media and AuditLog records.
- **ComplaintMedia:** Stores evidence photos.
 - *Fields:* type (BEFORE / AFTER), url, uploaded_by.
- **AuditLog:** An immutable ledger of every action taken on a complaint.
 - *Fields:* actor_id, previous_status, new_status, action, timestamp.

2. Key System Flows

Flow A: Complaint Submission & Triage

1. **Citizen** logs in and submits a complaint with a description, location, and a "BEFORE" photo.
2. The system creates a new Complaint record with the status SUBMITTED.

3. A **Receiving Officer** reviews the complaint on their dashboard and forwards it to the correct department. Status changes to FORWARDED.

Flow B: Department Assignment & SLA Tracking

1. The **Department Head** sees the forwarded complaint. Based on the category's sla_hours, a deadline is automatically calculated.
2. The Head selects a worker from their staff list and clicks "Assign". Status changes to ASSIGNED.
3. If the complaint remains open past the SLA deadline (e.g., 3 days), the Department Head can trigger a dept_warning (e.g., "Meet our office immediately"), which becomes highly visible on the Field Staff's dashboard.

Flow C: Resolution & Verification

1. The **Field Staff** worker acknowledges the assignment by clicking "Start Work". They input an expected visit date and a contact number for the citizen. Status changes to IN_PROGRESS.
2. Upon finishing the physical work, the Field Staff uploads an "AFTER" photo and marks it solved. Status changes to EVIDENCE_SUBMITTED.
3. The **Department Head** reviews the evidence. If acceptable, they approve it, changing the status to CITIZEN_VERIFICATION.
4. The **Citizen** sees the resolved complaint and "AFTER" photo. They click "Confirm Resolution", which finally moves the status to CLOSED.

3. API & Interface Design

The backend uses Next.js RESTful API Routes to handle client requests securely.

Core Endpoints:

- POST /api/auth/register & POST /api/auth/login
 - *Purpose:* Authenticates users, hashes passwords using bcrypt, and generates a JWT session token.
- GET /api/complaints
 - *Purpose:* Retrieves a list of complaints.
 - *Security:* The API reads the user's JWT role. Citizens only get their own records, Dept Heads get department-wide records, and Field Staff get assigned records.
- POST /api/complaints/[id]

- *Purpose*: The primary state-machine controller for a complaint.
- *Payload*: { action: string, data?: object } (Actions include: assign, start_work, submit_evidence, approve_evidence, confirm_resolution).
- *Behavior*: Verifies the user's role against the requested action. Updates the status column and automatically injects a record into the AuditLog table.
- GET /api/dashboard/stats
 - *Purpose*: Aggregates counts (e.g., Total Open, Overdue, Resolved) for rendering analytical charts on the dashboard.

4. Error Handling & Validation

To ensure system stability, errors are handled at both the client and server levels.

Server-Side (API) Error Handling

- **Authentication Errors (401 Unauthorized)**: If a user attempts to access an API route without a valid JWT, the server immediately aborts the request and returns a 401 status.
- **Authorization Errors (403 Forbidden)**: If a Field Staff member attempts an action reserved for a Department Head (like assigning a worker), the API catches the role mismatch and returns a 403 status.
- **Database Constraints (500 Internal Error)**: Prisma ORM enforces foreign-key constraints. If an API route attempts to assign a complaint to a non-existent user ID, Prisma throws a known error code. The API catches this in a try/catch block and returns a safe, sanitized 500 error message to the client, preventing database corruption or leaked stack traces.

Client-Side (UI) Error Handling

- **Form Validation**: The React frontend uses state constraints to prevent submission of incomplete data. For example, the "Submit" button on a complaint form remains disabled until a location and description are provided.
- **Graceful Degradation**: If an API request fails (e.g., network disconnect), the UI catches the rejected fetch Promise and displays a red Toast notification (e.g., *"Failed to update complaint. Please try again."*) rather than crashing the application.